

CERCO AO TRONO

Documentação Técnica Completa do Projeto

Versão: 1.1 — Junho de 2026

Motor: Unity 6 (URP — Universal Render Pipeline)

Linguagem: C#

Plataforma: Web e Mobile (Foco em Mobile)

EQUIPE DE DESENVOLVIMENTO

Augusto Zanin

João Dambros

João Vitor do Nascimento

Rafaela Silva

1. VISÃO GERAL DO PROJETO

1.1 Conceito

Cerco ao Trono é um jogo 2D de estratégia e física inspirado em *Angry Birds*, ambientado em um universo de fantasia medieval. O jogador assume o papel de um cobrador de impostos rebelde que financia uma revolução contra o Rei Áureo, o Ganancioso.

A mecânica central combina gestão de recursos (orçamento de ouro) com física destrutiva: o jogador seleciona tropas dentro de um budget fixo por fase e as lança contra as estruturas do castelo inimigo usando um sistema de estilingue estilo *Angry Birds*.

Campo	Valor
Título	Cerco ao Trono
Motor	Unity 6 (URP - Universal Render Pipeline)
Gênero	Estratégia / Puzzle baseado em física 2D
Plataforma	Web e Mobile (Foco em Mobile)
Linguagem	C#
Classificação	E10+ (Fantasia Medieval, Violência Leve)
Previsão	Maio de 2026
Equipe	Augusto Zanin, João Dambros, João Vitor do Nascimento, Rafaela Silva

1.2 Narrativa

Um cobrador de impostos real, cansado da tirania e dos gastos excessivos do Rei Áureo, o Ganancioso, decide usar seu conhecimento sobre as finanças do reino para financiar uma rebelião. O jogo começa com a invasão das vilas periféricas e termina com o cerco ao Castelo de Ouro.

Após a derrota do Rei no nível final, o castelo desmorona e o próprio Comandante assume o poder, tornando-se ironicamente o mais novo rei ganancioso.

1.3 Loop Principal de Jogo

- FASE DE PREPARAÇÃO:** O jogador analisa o castelo, gasta seu budget de ouro escolhendo tropas adequadas para o ataque.

2. **FASE DE EXECUÇÃO:** O jogador clica em Atacar. As tropas são spawned uma a uma no SpawnPoint. O jogador mira e dispara cada uma usando o estilingue (ou toca para ativar habilidades especiais).
3. **RESULTADO:** Se o Rei morrer → Vitória com cálculo de estrelas (1 a 3). Se todas as tropas forem usadas sem matar o Rei → Derrota.

2. ESTRUTURA DE CENAS

IMPORTANTE: As cenas devem estar no Build Profiles (File -> Build Profiles no Unity 6) na ordem correta para que SceneManager.LoadScene funcione pelo nome.

2.1 Cena 0 - MainMenu

Tela inicial do jogo. Contém o título e dois botões de navegação.

Elemento	Tipo	Descrição
Canvas	Canvas (Screen Space Overlay)	Container principal da UI
Título	TextMeshProUGUI	Texto: Cerco ao Trono
Botão Jogar	Button + TMP	Chama OnPlayPressed() -> carrega MapaFases
Botão Créditos	Button + TMP	Chama OnCreditsPressed() -> PENDENTE de implementação
MainMenuUI	Script (C#) no Canvas	Gerencia navegação entre cenas

2.2 Cena 1 - MapaFases

Tela de seleção de fases. Exibe botões para cada fase disponível e um botão de voltar.

Elemento	Tipo	Descrição
Canvas	Canvas (Screen Space Overlay)	Container principal da UI
Texto Mapa	TextMeshProUGUI	Título da tela
Botão Fase 1	Button + TMP	Chama LoadFase("Fase1") no MapaFasesUI
Botão Voltar	Button + TMP	Chama BackToMenu() no MapaFasesUI
MapaFasesUI	Script (C#) no Canvas	Gerencia carregamento de fases

2.3 Cena 2 - Fase1 (arquivo: SampleScene)

Cena principal de gameplay. Toda a lógica do jogo acontece aqui.

Hierarquia Completa de Objetos

```
Fase1 (SampleScene)
├─ Main Camera [Orthographic, Size ~8-10, ajustar para ver campo completo]
├─ Global Light 2D [Iluminação URP]
├─ Ground [Verde, Rigidbody2D Kinematic, BoxCollider2D, Tag: Ground]
├─ Cubo (vários) [Blocos marrons, Rigidbody2D Dynamic, BoxCollider2D]
├─ King [Quadrado amarelo, Rigidbody2D Dynamic, CircleCollider2D, King.cs]
├─ GameManager [GameObject vazio, GameManager.cs]
├─ SpawnPoint [GameObject vazio, posicionado à esquerda da tela]
├─ Canvas [Screen Space Overlay, Scale With Screen Size 1920x1080]
│   └─ Panel [Transparente, stretch total, PreparationUI.cs]
│       ├── GoldText [TMP, âncora top-left, exibe 'Gold: 1000']
│       ├── TroopList [GameObject vazio, Vertical Layout Group, cards de tropas]
│       ├── AttackButton [Button + TMP, âncora bottom-right, texto 'Atacar']
│       └─ UndoButton [Button + TMP, acima do AttackButton, texto 'Desfazer']
│   └─ VictoryPanel [Panel oculto por padrão, fundo verde semi-transparente]
│       ├── Text VITORIA [TMP, fonte grande, centralizado]
│       ├── Star1/2/3 [GameObjects de estrela acesos/apagados por GameOverUI]
│       ├── StarsLabel [TMP - exibe 'X / 3']
│       └─ MapaButton [Button -> GameOverUI.BackToMenu()]
│   └─ DefeatPanel [Panel oculto por padrão, fundo vermelho semi-transparente]
│       ├── Text DERROTA [TMP, fonte grande, centralizado]
│       ├── TentarButton [Button -> GameOverUI.RetryLevel()]
│       └─ MenuButton [Button -> GameOverUI.BackToMenu()]
│   └─ KingHealthBarBG [UI Image, âncora top-center, 600x30px, vermelho escuro]
│       └─ KingHealthBarFill [UI Image filho, stretch total, vermelho brilhante, pivot X=0]
└─ EventSystem [OBRIGATÓRIO - sem ele nenhum botão de UI funciona]
```

3. SCRIPTS - REFERÊNCIA COMPLETA

NOTA DE LOCALIZAÇÃO: Todos os scripts estão em Assets / Scripts /. Nunca mova scripts sem atualizar as referências nos prefabs e na cena.

3.1 Troop.cs - Classe Base Abstrata

Todas as tropas herdam desta classe. Gerencia a mira estilo estilingue, o preview de trajetória pontilhada e o lançamento por física. Também controla a limpeza do campo após o projétil parar.

Campos Públicos

Campo	Tipo	Padrão	Descrição
troopName	string	-	Nome de exibição da tropa
goldCost	int	-	Custo em ouro para adicionar à fila
icon	Sprite	null	Ícone exibido no card da UI
launchForce	float	15f	Multiplicador de velocidade de lançamento
maxDragDistance	float	2.5f	Distância máxima de arraste do estilingue
trajectoryDots	int	30	Quantidade de pontos no preview de trajetória
trajectoryTimeStep	float	0.1f	Intervalo de tempo entre pontos da trajetória
dotPrefab	GameObject	null	Prefab do ponto de trajetória (assign: Trajectory)

Propriedades

- **IsReadyToLaunch (bool):** definida pelo GameManager via Activate(). Enquanto false, nenhum input é aceito pela tropa.
- **HasLaunched (bool):** true após o lançamento. Usado pelo GameManager para detectar quando spawnar a próxima tropa.

Métodos Virtuais para Override

Método	Quando usar override	Exemplo de uso
Launch(Vector2 velocity)	Quando o lançamento é completamente diferente do padrão.	BestaGigante: não voa, apenas dispara projéteis.
OnLaunched(Vector2 velocity)	Para efeitos adicionais após lançamento normal.	Animações, partículas, habilidades especiais.
OnHit(Collision2D col)	Para dano ou efeito especial ao colidir.	Explosão, congelamento, fogo.

Como o Estilingue Funciona

1. Jogador clica próximo à tropa (distância < 0.8 unidades Unity).
2. Arrasta o mouse na direção oposta ao alvo.
3. Ao soltar: $velocity = -dragOffset * launchForce$.
4. Preview de trajetória usa equação cinemática: $pos = startPos + v*t + 0.5*g*t^2$.
5. Ao lançar: $gravityScale = 1$, constraints liberadas, $rb.linearVelocity = velocity$.

ATENÇÃO UNITY 6: Sempre use `rb.linearVelocity` (não `rb.velocity`). Esta API mudou no Unity 6 e o código antigo gera avisos de obsolescência (*deprecated*).

3.2 Catapult.cs - Projétil Básico

Projétil pesado padrão. Sem habilidade especial - apenas dano por impacto de física. A tropa mais simples e intuitiva do jogo.

Campo	Tipo	Padrão	Descrição
<code>impactRadius</code>	float	0.5f	Raio visual do impacto (apenas cosmético por enquanto)

3.3 UnicornKamikaze.cs - Unicórnio Explosivo

Unidade focada em investidas retas e explosões devastadoras.

Campo	Tipo	Padrão	Descrição
<code>explosionRadius</code>	float	2f	Raio da explosão em unidades Unity
<code>explosionForce</code>	float	20f	Força aplicada aos objetos no raio
<code>chargeSpeed</code>	float	14f	Velocidade horizontal da corrida
<code>explosionEffectPrefab</code>	GameObject	null	Partícula de explosão (opcional)

Comportamento Detalhado

- Não usa estilingue - qualquer clique após `Activate()` dispara a corrida imediatamente.
- `gravityScale` = 0 durante a corrida - corre perfeitamente horizontal sem cair.
- Ao colidir: `Physics2D.OverlapCircleAll` encontra todos os objetos no raio.
- Cada objeto recebe força proporcional à distância (mais perto = mais força).
- Destrói a si mesmo após 0.1s (delay para partícula de explosão aparecer primeiro).

3.4 BestaGigante.cs - Disparo Múltiplo

Arma estática que faz um disparo em leque de múltiplos projéteis menores.

Campo	Tipo	Padrão	Descrição
arrowCount	int	3	Quantidade de projéteis disparados simultaneamente
spreadAngle	float	15f	Ângulo em graus entre cada projétil
arrowForce	float	20f	Força aplicada a cada projétil ao disparar
arrowPrefab	GameObject	null	Prefab do projétil disparado

Comportamento Técnico

- Override completo de `Launch()` - não chama `base.Launch()` para o corpo não voar.
- Calcula ângulo base a partir do vetor de velocidade do arraste do estilingue.
- Distribui `arrowCount` projéteis simetricamente ao redor do ângulo base.
- Cada projétil é instanciado com `Rigidbody2D + Collider` e recebe `AddForce`.
- O componente `Troop` é removido de cada projétil para evitar conflito de inicialização.
- O corpo da besta é destruído logo após os disparos.

SETUP OBRIGATÓRIO: Atribuir `arrowPrefab` no Inspector do prefab `BestaGigante`. Use o prefab `Catapulta` como placeholder até a arte estar pronta.

3.5 GoblinMineiro.cs - Ataque Subterrâneo

Unidade tática que viaja por baixo da terra para explodir alvos específicos por baixo.

Máquina de Estados (5 Estados)

Estado	O que acontece
Idle	Aguarda clique do jogador. Apenas cliques à direita do goblin são aceitos.
DiggingDown	Move verticalmente para baixo até atingir <code>undergroundY</code> (<code>groundY - depth</code>).
Underground	Move horizontalmente até o X do ponto clicado, mantendo a profundidade.
SurfacingUp	Move verticalmente para cima até <code>groundY + surfaceOffset</code> .
Exploding	Detona a explosão e destrói o objeto.

SETUP CRÍTICO: O objeto `Ground` na cena DEVE ter a tag `'Ground'` para o `GoblinMineiro` detectar o nível do chão corretamente.

3.6 King.cs - O Alvo Principal

O Rei é o alvo que o jogador precisa destruir para vencer a fase.

- Se atingido diretamente por uma Troop: recebe dano total.
- Se o objeto colisor não tem Troop (blocos, debris): $\text{dano} = \text{crushDamage} * (\text{força} / 10)$.
- Ao chegar em 0 HP: chama `GameManager.Instance.CheckForVictory()` e se destrói após 0.5s.

ATENÇÃO: O King NÃO usa a tag 'King'. Ele é encontrado via `FindAnyObjectByType<King>()`. Não adicione tags ao King pois quebraria a busca no `GameManager`.

3.7 GameManager.cs - Gerenciador Central

Singleton que controla o estado global do jogo: budget de ouro, fila de tropas, spawn sequencial, sistema de estrelas e acionamento das telas de fim de jogo.

Sistema de Estrelas (Pontuação)

- **1 estrela:** Rei morreu (condição mínima de vitória).
- **2 estrelas:** Rei morreu E ouro gasto $\leq 50\%$ do budget inicial.
- **3 estrelas:** Rei morreu E apenas 1 ou 2 tropas lançadas no total.

Fluxo de Spawn de Tropas (Passo a Passo)

1. Jogador clica no card → `TryAddTroop()` deduz ouro e adiciona o prefab à `troopQueue`.
2. Jogador clica Atacar → `goldSpent` é calculado → `StartExecution()` é chamado.
3. `SpawnNextTroop()` instancia `troopQueue[index]` na posição do `SpawnPoint`.
4. `activeTroop.Activate()` é chamado - libera input na tropa.
5. `WaitForTroopToFinish()` aguarda `HasLaunched = true` → espera 2 segundos → chama `SpawnNextTroop()` novamente.
6. Quando todas as tropas foram lançadas → `CheckForDefeat()` aguarda 3 segundos de tolerância física.
7. Se `King.health <= 0` → Vitória instantânea via `GameOverUI.ShowVictory(stars)`.

3.8 a 3.11 Outros Scripts de UI

- **PreparationUI.cs:** Controla o painel de compras. Contém a classe interna `TroopData` para configurar os dados das tropas direto no Inspector. Possui o botão Desfazer (`OnUndoPressed()`) que estorna o custo da última tropa comprada.

- **TroopCard.cs:** Componente visual do card de compra. Para evitar que os textos filhos bloqueiem os cliques no botão pai, o método `Setup()` desativa automaticamente o `raycastTarget` de todos os `TextMeshProUGUI` filhos.
- **GameOverUI.cs:** Fica atrelado ao Canvas principal. Controla a exibição e os botões dos painéis de vitória e derrota.
- **KingHealthBar.cs:** Atualiza a barra de vida do topo da tela. Utiliza a escala horizontal do objeto (`localScale.x`) em vez de `fillAmount` para garantir compatibilidade perfeita com sprites do tipo *UISprite* sob o pipeline URP do Unity 6.

4. PREFABS - REFERÊNCIA COMPLETA

Prefab	Componentes	Configuração Obrigatória
Catapulta	SpriteRenderer, Rigidbody2D Dynamic, CircleCollider2D, Catapult.cs	<code>dotPrefab = Trajectory</code>
UnicornKamikaze	SpriteRenderer, Rigidbody2D Dynamic, CircleCollider2D, UnicornKamikaze.cs	<code>dotPrefab = Trajectory</code> (opcional)
Trajectory	SpriteRenderer (Círculo Branco Pequeno)	Escala pequena (~0.15). Usado como ponto de preview de trajetória.
TroopCard	Image, Button, TroopCard.cs, TextMeshProUGUI (filhos)	Mapear filhos <code>NameText</code> e <code>CostText</code> no Inspector.

5. TROPAS E BALANCEAMENTO

5.1 Status Atual de Implementação

Tropa	Script	Custo GDD	Custo Atual	Status
Catapulta	Catapult.cs	200	100	IMPLEMENTADA - custo errado na Engine
Unicórnio Kamikaze	UnicornKamikaze.cs	250	100	IMPLEMENTADA - custo errado na Engine
Besta Gigante	BestaGigante.cs	150	150	IMPLEMENTADA - sem arte final
Goblin Mineiro	GoblinMineiro.cs	100	100	IMPLEMENTADA - sem arte final
Dragão de Fogo	-	300	-	NÃO IMPLEMENTADA
Mago Gelo	-	120	-	NÃO IMPLEMENTADA
Mago Raio	-	120	-	NÃO IMPLEMENTADA

PENDÊNCIA: Corrigir goldCost de Catapult para 200 e Unicórnio para 250 diretamente no Inspector do Panel -> PreparationUI -> Available Troops.

6. CONFIGURAÇÕES DO UNITY

6.1 Notas Críticas do Unity 6

- Use sempre `rb.linearVelocity` (NÃO `rb.velocity`).
- Use `FindAnyObjectByType<T>()` (NÃO os antigos métodos obsoletos).
- O **EventSystem** DEVE existir na cena para que a UI responda a qualquer interação.
- Canvas configurado com *Scale With Screen Size* (Referência: 1920x1080).
- Filhos TMP de botões devem ter `raycastTarget` desativado.

6.2 Build Profiles - Ordem das Cenas

1. Index 0: MainMenu

2. Index 1: MapaFases

3. Index 2: Fase1

7. PROBLEMAS CONHECIDOS E PENDENCIAS

#	Problema / Funcionalidade	Prioridade	Como Resolver / Notas
1	goldCost incorreto na Engine	ALTA	Ajustar no Inspector da PreparationUI.
2	Arte real (sprites) ausente	ALTA	Substituir primitivos Unity por sprites reais do projeto.
3	Dragão e Magos ausentes	ALTA	Criar scripts, arte e prefabs do zero.
4	Música e Efeitos Sonoros	ALTA	Nenhum som implementado no momento.
5	Persistência de Estrelas	MÉDIA	Salvar estrelas conquistadas entre sessões de jogo (PlayerPrefs).

8. CHECKLIST FINAL DE CONFIGURAÇÃO (QA)

- [] Na cena Fase1: GameManager tem troopSpawnPoint e gameOverUI atribuídos.
- [] Na cena Fase1: Panel tem PreparationUI com todos os campos preenchidos.
- [] Na cena Fase1: GameOverUI tem victoryPanel, defeatPanel e star1/2/3 atribuídos.
- [] Na cena Fase1: KingHealthBarBG tem KingHealthBar.cs com fillBar atribuído.
- [] Na cena Fase1: EventSystem existe na hierarquia.
- [] Ground tem a tag exata '**Ground**'.
- [] BestaGigante prefab tem o arrowPrefab atribuído corretamente.

Cerco ao Trono - Documentação Técnica v1.1 - Junho de 2026 - Augusto Zanin, João Dambros, João Vitor do Nascimento, Rafaela Silva